Listas

Una lista es una colección ordenada, mutable y, en el caso de Python, heterogénea de elementos. Se definen con corchetes `[]`.

```
vacia = []
numeros = [1, 2, 3, 4, 5]
mixta = [1, "Hola", 3.14, True]
```

Indexación y slicing

Puedes acceder o modificar los elementos por su índice.

> RECUERDA QUE SE EMPIEZA EN 0!

```
colores = ["rojo", "cian", "morado", "verde", "amarillo", "negro"]
print(colores) # ['rojo', 'cian', 'morado', 'verde', 'amarillo', 'negro']
print(colores[0]) # rojo
print(colores[-1]) # negro
```

Slicing

Se pueden crear sublistas haciendo uso de `[inicio:fin:salto]`, el funcionamiento se parece al range pero en este caso se puede hacer sobre colecciones indexadas (en este caso se hace listas, pero veremos que se puede usar sobre cadenas de texto).

```
colores = ["rojo", "cian", "morado", "verde", "amarillo", "negro"]
print(colores[0:2]) # ['rojo', 'cian']
print(colores[:2]) # ['rojo', 'cian']
print(colores[1:-1]) # ['cian', 'morado', 'verde', 'amarillo']
print(colores[::-1]) # ['negro', 'amarillo', 'verde', 'morado', 'cian',
'rojo']
```

Modificación de listas

```
colores = ["rojo", "cian", "morado", "verde", "amarillo", "negro"]
colores[1] = "azul"
print(colores) # ['rojo', 'azul', 'morado', 'verde', 'amarillo', 'negro']
```

```
```

```
colores = ["rojo", "cian", "morado", "verde", "amarillo", "negro"]
colores.append("blanco")
colores.insert(1, "rosa")
colores.remove("rojo")
colores.pop() # 'blanco'
colores.pop(0) # 'rojo'
colores.extend(["aquamarine"])
```

- `append(x)` añade al final el elemento `x` al final de la lista
- `extend(x)` agrega al final de la lista los elementos que componen la colección `x`.
- `insert(n, x)` agrega en el indice `n` de la lista el elemento `x`, si esta posición ya estaba ocupada, desplaza el elemento que lo ocupara y todos los consecutivos.
- `remove(x)` elimina el elemento `x` de la lista.
- `pop()` si no se determina ninguna posición, saca de la lista el último elemento de la lista y lo retorna. En caso de pasarle un número por parámetro, saca el elemento en esa posición de la lista y mueve el resto.

Operaciones básicas

```
a = [1, 2, 3]
b = [4, 5]
print(a + b) # [1, 2, 3, 4, 5] (concatenar)
print(a * 2) # [1, 2, 3, 1, 2, 3] (repetir)
print(len(a)) # 3
```

Verificación y búsqueda

```
a = [1, 2, 3]
3 in a # True
10 not in a # True
a.index(2) # 1 (posición del valor)
a.count(3) # 1 (veces que aparece)
```

Recorrer listas

```
for elemento in ["A", "B", "C"]:
    print(elemento)
```

```
colores = ["rojo", "cian", "morado", "verde", "amarillo", "negro"]
for i in range(len(colores)):
    print(i, nombres[i])
```

List Comprehension

El list comprehension es una forma rápida y compacta de crear listas nuevas a partir de una secuencia (como un range(), otra lista, etc.).

```
[expresión for elemento in iterable if condición]
```

- `expresión` → lo que se va a añadir a la nueva lista.
- `elemento` → cada valor del iterable.
- `if condición` → (opcional) filtra qué elementos incluir.

```
cuadrados = [x**2 for x in range(5)]
# [0, 1, 4, 9, 16]

pares = [x for x in range(10) if x % 2 == 0]
# [0, 2, 4, 6, 8]

result = ["par" if x % 2 == 0 else "impar" for x in range(5)]
print(result) # ['par', 'impar', 'par', 'impar', 'par']
```